



Chapter 1

Introduction

Computer science is the science that deals with the treatment of information by means of automatic procedures. It has multiple objectives, including the study of computation at a logical level and the several possible practical techniques for its implementation and application in automated electronic systems, i.e., computers. One of the focal points of computer science is *problem solving*, i.e., the act of defining a problem and then developing and implementing a solution. In this context, a *problem* can be defined as a task that has to be fulfilled automatically, and solving a problem typically implies the design and development of an *algorithm*. An algorithm is a process, or method, that solves a problem by means of a finite set of well-defined and unambiguous actions, each of which can be mechanically executed in a finite amount of time. At the bottom of the process stands the implementation, which consists in the development of a computer *program*, i.e., the encoding of an algorithm in a programming language, which can be directly compiled and/or executed on a computer. A problem can typically be expressed by means of a set of data, called *inputs*, that the algorithm, and consequently the program that implements it, can use. The execution of the program on a computer finally produces the solution to the problem, which can be the value of a variable or set of variables, or any other type of information we were looking for, called the *output*.

Example 1.1. Arguably one of the simplest problems that can be solved using a computer is the following one:

Find the maximum of a finite sequence of numbers.

An algorithm to solve this problem can be described by the following actions:

1. Define a numeric variable called *max*;
2. Assign to *max* the value of the first number in the sequence;
3. For each value *x* in the sequence:
 - 3.1. if *x* is larger than the value currently contained in variable *max* then change the value of *max*, so that it becomes equal to *x*;
4. return the value of variable *max*;

Such an algorithm can be executed manually for small sequences, but it clearly needs automatic computation for large sequences. In order to execute the algorithm automatically on a computer, it needs to be encoded in a programming language. For instance, a possible program that implements the previous algorithm can be represented by the following Java method:

```
public static int findMax(int[] inputSequence) {  
    int max = inputSequence[0];  
    for (int i = 1; i < inputSequence.length; i++) {  
        if (inputSequence[i] > max)  
            max = inputSequence[i];  
    }  
    return max;  
}
```

In this example, the input (i.e., the data that characterize the problem) is the given sequence of numbers and the output (i.e., the sought for result) is a value, the maximum of the sequence. Encoding an algorithm into a computer program typically implies a number of implementation choices, such as, in this case, the data types of the numeric values contained in the sequence (integer numbers in the implementation given above), the way of representing the sequence of numbers (an array of integer values, in this case), and the way of receiving inputs and returning outputs (parameter passage of Java methods and `return` Java statement, respectively).

From this example, it should be clear that, in the context of computer science, a problem is generally not dependent on the particular data, or instance, used, but it is instead a much more general concept, which actually *abstracts* from the data used in the particular instance. The above problem could, in fact, be reformulated as “Develop a method to find the maximum of *any* finite sequence of numbers”, and this is exactly what the reported algorithm and program do. In particular, one should remark that the Java method `findMax` can be used for finding the maximum of *any* array of integer values, independently of its length (provided that it is not longer than the longest sequence that can fit in the memory of the computer we are using) and of the particular values it contains.

Furthermore, it should not be hard to convince oneself that, apart from some implementation choices like the ones discussed above, coding an algorithm into a computer program is broadly a mechanical task. In fact, each particular action, or group of actions, contained in an algorithm has an immediate correspondence to one or more instructions of a programming language. On the other hand, developing an algorithm able to solve a problem is, typically, a very *creative* task, for which no formal rules exist in general. In traditional computer science, the design of an algorithm, which actually corresponds to the development of the “strategy” to solve the problem, is completely manual and assigned to human beings.

1.1 Motivation for Intelligent Systems

Intelligent systems can be broadly defined as systems that have the ability to solve problems autonomously, i.e., with reduced or even no intervention from humans. The motivation for intelligent systems is straightforward: the traditional computational method described previously, where the design of an algorithm is assigned to a human being, fails in many cases. This can happen, depending on the situation, because it is extremely hard, or even impossible, for a human being to conceive an algorithm for a problem, or because the algorithms that we are able to design for particular problems are so inefficient that it is impossible to execute their implementations in a reasonable amount of time, even on the most powerful existing computers. As a first and simple example, one may think of the following problem: assume that you are the owner of a store, and a person enters your store. The problem consists in giving a reliable answer to the following question:

Will that person be a good client?

Or, in other words: will that person buy enough products sold in your store, so that you will achieve a profit? Remembering that an algorithm is a precise sequence of formal actions that allow us to solve a general problem (in other words, the algorithm should potentially work for *any* person entering the store), we challenge the reader to design an algorithm for this problem. We believe that we can state, with a high degree of confidence, that such a design is impossible for a human being.

Let us now consider another example. Given a set of photographs picturing human faces, answer the following question:

Who is the person that is pictured in this photo?

In this case, a human being may possibly imagine an algorithm, mainly based on somatic characteristics of the face, like the color of the hair, the color of the eyes, the distance between the eyes, the position, shape and size of nose and mouth, etc. However, it is clear that it is possible to present to a human being two photos picturing the faces of two different persons with absolutely identical somatic characteristics, and, even in those cases, the human brain is able to distinguish between the two pictured persons. Probably no handmade algorithm will ever have the discernment ability that is typical of the human brain, and so probably none of them will ever solve this problem in a satisfactory way. Indeed, practically the totality of the face recognition systems existing nowadays heavily rely on intelligent systems, instead of traditional computational methods.

Let us consider a third example. The problem consists in answering the following question:

What is the right mix of molecules that can be used to create a new drug?

For instance, we may be interested in creating a new medicine with some particular characteristics, or able to cure a new disease. In this case, it is possible, and arguably even “easy”, for a human being to conceive an algorithm. For instance, the algorithm may consist in trying, one by one, all the possible combinations of known molecules

and testing them, to see whether they obtain the sought for effect. This algorithm can, in principle, solve the problem perfectly, but it has a major flaw: the number of molecules existing in one cubic centimeter is estimated as being approximately equal to 2.7×10^{19} . Even without dwelling on the calculations required to estimate the number of possible combinations of existing molecules, it should not be hard to convince oneself that testing all these combinations in acceptable time is impossible.

As a fourth example, one may imagine the problem of writing a program able to drive a robot in a 3D space, so that the robot is able to efficiently accomplish complex tasks, independently of the characteristics of the particular space. To fix ideas, one may consider the practical case of self-driving vehicles. Also in this case, it is not hard to convince oneself that the number of possible situations that the robot may have to deal with is so high that any handcrafted algorithm is probably destined to fail. In fact, self-driving vehicles are based on intelligent systems.

The above list of examples could continue indefinitely. For all these problems, either it is impossible for a human being to conceive an algorithm or the algorithms that can be developed are not executable in practice. Generally speaking, none of these problems can be solved using the traditional computational method. At the same time, with today's proliferation of data, these types of problems are becoming more and more numerous and, thanks to improvements in technology, we are every day more and more interested in solving those problems. So, what to do? The idea is to design computational systems that have the ability to autonomously *learn* to solve problems. This is the objective of the area of study, particularly popular and active nowadays, called *Machine Learning*. The idea of building machines able to learn is not new. Alan Turing was probably the first person to describe computer intelligence as "a machine that can learn from experience" in a public lecture in London, in 1947, going as far as stating that "the possibility of letting the machine alter its own instructions provides the mechanism for this". However, these ideas remained unpublished, only to be reinvented later. Early work can be broadly dated back to the studies of Samuel in the late 1950s [Samuel, 1959], later revisited and deepened by Mitchell in the 1990s [Mitchell, 1997]. According to Mitchell, Machine Learning is:

"the study of algorithms able to automatically improve by means of experience"

This definition is significant for at least two reasons. First of all, Mitchell puts the accent on algorithms, actually raising the bar on the objective of overcoming the limitations of traditional computing, which entirely assigns the design of algorithms to humans. Secondly, Mitchell defines the concept of learning as improvement by means of experience. In this way, Mitchell gives a clear idea of the general functioning of a learning computational system: it has to be based on an iterative process in which, at each iteration, either a new entire solution or a piece of an existing solution is proposed, based on the experience gathered in the previous iterations. Key in all this process is, once again, the idea that solutions to complex problems should be built with very limited or even no human intervention. The only task for humans now is to give the system a very precise description of the problem, and no hint on

how to solve it, while the solution should be completely generated in an automatic way by the computational system.

1.2 Intelligent Systems and Bioinspired Algorithms

A very large number of techniques that can be considered intelligent exist nowadays, and a significant part of those techniques draw inspiration from the idea that learning is a characteristic that is typical of living beings. So, those techniques are inspired by the biology of some living beings with the objective of, in some way, mimicking their ability to learn. When this happens, we talk about *bioinspired* intelligent systems, or simply bioinspired algorithms. The best known are:

- *Artificial Neural Networks*, which are inspired by the structure of the human brain, and try to simulate, although in an elementary way, its functioning.
- *Evolutionary Algorithms*, which are inspired by the Theory of Evolution of Charles Darwin [Darwin, 1859], and have the objective of evolving solutions using processes that, although simplified, are similar to those that are responsible for the evolution of species.
- *Swarm Intelligence*, which draws inspiration from the collective intelligence arising from the collaboration of animals in groups, like for instance ant colonies, bird flocking, hawks hunting, animal herding, bacterial growth, fish schooling and microbial intelligence.
- *Fuzzy Systems*, which simulate the ability of human beings to deal with concepts such as uncertainty or imprecise information, an ability that is crucial for learning general behaviors.
- *Local Search Systems*, whose biological inspiration can arguably be imagined as coming from their ability to simulate the sense of orientation of living beings.

All these approaches share one important characteristic: they have the ability to return an approximate, or imprecise, solution to problems whenever they are not able to find a perfect one. This characteristic is typical of the set of techniques that are identified with the name *Soft Computing* [Tettamanzi and Tomassini, 2001]. This feature can be very important in many situations for at least the following reasons: first of all, problems exist that are so complex that the idea of finding a perfect solution is purely utopian. In those cases, obviously, it is better to have an approximate solution than to not have any solution at all. Secondly, there are cases in which an approximate solution can be as good as a perfect one. One may, for instance, imagine a problem consisting in driving a robot in a room, following a path from a given starting point *A* to a given destination *B*, avoiding obstacles that are positioned in the room. In this context, it is obvious that no particular path is required, as any path that allows the robot to go from *A* to *B* avoiding obstacles is exactly as good as all the others that have this characteristic. Last but not least, having the ability to return approximate solutions can be crucial for improving the

generalization ability of a system, given that a perfect solution may be affected by *overfitting*, a key concept of Machine Learning discussed in Chapter 5.

This book introduces various bioinspired intelligent systems and also some intelligent systems that are not bioinspired. In particular, we will discuss two types of local search algorithms, Hill Climbing and Simulated Annealing, two types of evolutionary algorithms, Genetic Algorithms and Genetic Programming, one swarm intelligence algorithm, Particle Swarm Optimization, and various types of artificial neural networks. Among the systems that are not bioinspired, we will present Decision Trees, Random Forests, Support Vector Machines, Bayesian Classifiers and others.

1.3 Applications of Intelligent Systems

Intelligent systems are used nowadays in so many applications, both in industry and academia, that listing them all is inconceivable. However, in this section, we try to give an incomplete and limited list of possible applications, just to give the reader an idea of the abundance and breadth of possible areas where intelligent systems are successfully employed.

- *Robotics*. Optimization of paths and movements of robots, treatment of signals coming from sensors and other sources, and many other related applications are nowadays a reality, and they have led, for instance, to the development of self-driving vehicles and autonomous appliances.
- *Engineering*. The automatic synthesis of integrated electronic circuits and the management of mechanical systems, such as appliances, trains or assistance devices, are only few examples of the numerous engineering applications that are nowadays strongly based on intelligent systems.
- *Biology*. From the study of DNA and the human genome, to the modeling of complex biological systems like gene regulatory networks or viruses, and many other applications, intelligent systems have played a central role in recent developments in biology.
- *Chemistry*. The automatization of the drug discovery process, aimed at designing, developing and commercializing new medicines, and the study and the optimization of the 3D structure of molecules and their properties are only a few examples of the numerous applications of chemistry in which intelligent systems have been applied successfully.
- *Medicine*. The analysis and mining of the vast amounts of data that arise in radiomics, genomics, epigenomics and proteomics, together with clinical data and many other forms of data are nowadays analyzed by intelligent systems with the objective of developing predictive models for diagnosis, prognosis and therapy, supporting decision making in medicine.
- *Economics and Business*. Simulation and prediction of stock market trends, risk management, decision support in companies and organizations, and pricing are

only some of the numerous applications in which intelligent systems are applied in economics and business nowadays.

- *Marketing*. Optimization of marketing campaigns, optimization of the placement of goods in the supermarket shelves and design of marketing strategies are performed successfully using intelligent systems in several companies and organizations.
- *Security*. Traffic control, face recognition, identity checking and tracking are tasks that are automated and optimized by means of intelligent systems in many situations nowadays.
- *Computational Security*. Cryptography, identification of malware and protection of computational systems, from large to small scale, by means of intelligent systems are a reality.
- *Society*. Predicting debt problems in families, the school performance of children, or inclination towards gambling addiction, drug addiction or other types of addiction are currently performed by intelligent systems in several situations, facilitating early intervention and social support.
- *Pervasive Computing*. Computers are everywhere nowadays. For instance, houses equipped with domotics, smart appliances and interactive facilities, where intelligent systems obviously play a crucial role.

1.4 Objectives and Limits of This Book

The list of possible applications of intelligent systems presented above could continue indefinitely. The conclusion is straightforward: intelligent systems are general-purpose methods that can be used with success in practically all existing application areas. With this perspective, it is clear that it would be presumptuous, and ultimately impossible, to try to discuss all possible applications in one book. Instead, in this book we have decided to not enter into the details of any application, except for some examples in some specific cases. Instead, the book focuses on computational methods, with the objective of providing readers with the necessary methodological tools and competencies that can hopefully allow them to tackle complex applications in the future.

The book has two ambitions: first, remaining general enough to be independent of any one application, but at the same time concrete enough to promote the future use of intelligent systems in any such application. Second, disseminating to readers, in particular students or young researchers, the message that intelligent systems is still a young area, where many improvements are still possible, and often necessary. Research in this area is, today more than ever, in demand, and this book aims to be an incentive for young, enthusiastic and dynamic minds to express and develop new ideas, to actively contribute to the field.

Of course, being general (i.e., independent of applications) does not have to mean being abstract or insubstantial. The book is supposed to be the backbone of the didactic material of a university course. The numerous examples contained in the

book, as well as the necessary practical classes in the course, are the means to maintain an appropriate level of concreteness, in a discipline that is naturally at the frontier of research. However, it is our conviction that, in order to successfully apply intelligent computational methods at an industrial level, a deep expertise is needed which involves not only a “final user” knowledge of the needed technicalities to apply those methods, but also a deep understanding and possession of their inner functioning and dynamics. This is the type of knowledge that this book has the ambition to transfer.

The book will not present any programming language or environment for intelligent systems, for at least two reasons. First, for reasons of space: the book would otherwise become unreasonably long. Second, because the most popular technologies and programming environments evolve and change rapidly, and any one of them would shortly become outdated and would be replaced by others.

1.5 Organization of the Book

In Section 1.1, we identified the main motivation for intelligent systems in relation to complex problems, for which the manual design of an algorithm is either extremely hard or impossible. Many of those problems exist, but undoubtedly a large part of them can be categorized into two main classes of problems that, although strongly interrelated, have important differences: *optimization* problems and *machine learning* problems. The main organization of the book reflects this categorization. The book is, in fact, partitioned into two main parts: Part I, devoted to computational intelligence methods for optimization, and Part II, dedicated to Machine Learning (ML).

Part I is organized as follows:

- Chapter 2 introduces the concept of optimization and optimization problems, discussing some fundamental concepts, and also introducing some computational intelligence algorithms which can be categorized as local search algorithms, such as Hill Climbing and Simulated Annealing.
- Chapter 3 presents a population-based method belonging to the field of evolutionary algorithms, called Genetic Algorithms, deepening its functioning and discussing some advanced topics.
- Chapter 4 tackles another population-based method, belonging to the field of swarm intelligence called Particle Swarm Optimization.

Part II is organized as follows:

- Chapter 5 introduces Machine Learning, with its definitions and general concepts, and presents some simple algorithms.
- Chapter 6 discusses Decision Trees.
- Chapter 7 presents Artificial Neural Networks, and some of their variants for supervised ML.

- Chapter 8 tackles Genetic Programming, the second flavor of evolutionary algorithm discussed in this book.
- Chapter 9 presents Bayesian learning and some of its supervised ML algorithms.
- Chapter 10 introduces Support Vector Machines.
- Chapter 11 discusses Ensemble Methods.
- Finally, Chapter 12 focuses on Unsupervised Machine Learning, introducing some of the best-known clustering algorithms.